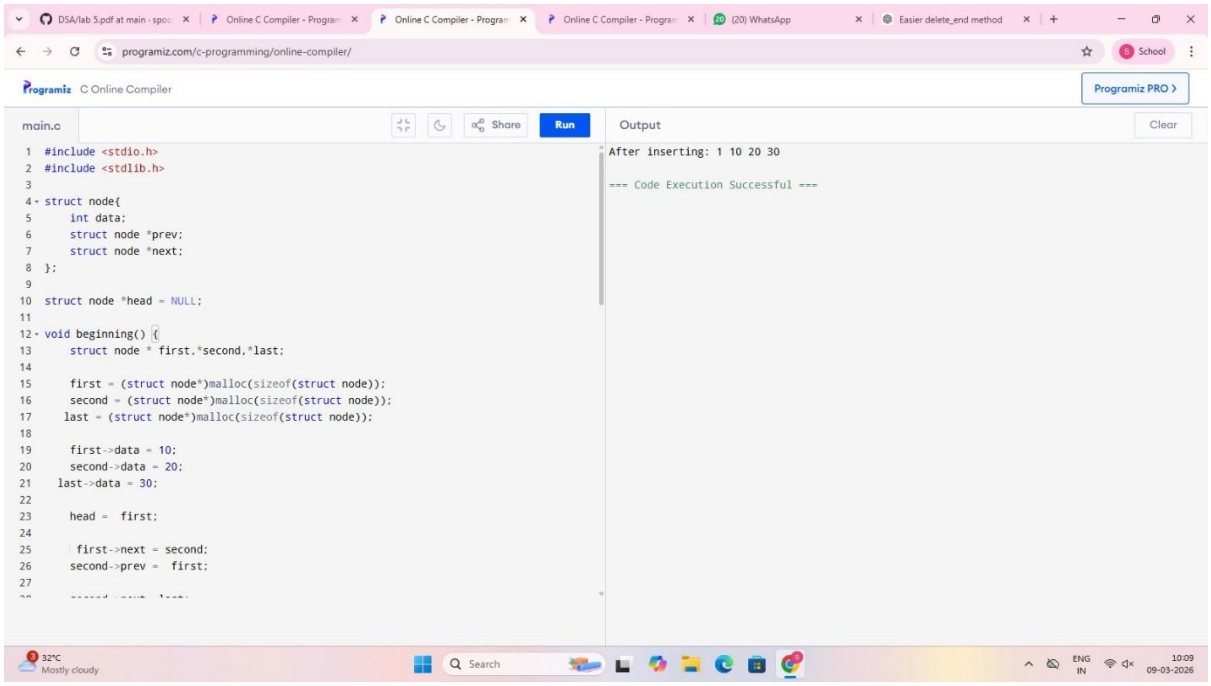
Doubly circular linked list

Insertion at beginning



```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node * first,*second,*last;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
   last = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
  last->data = 30;

    head =  first;

     first->next = second;
    second->prev =  first;
```

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

```c
    second->prev =  first;

    second->next =last;
last->prev = second;

   last->next = head;
    head->prev =last;
}

void insertStart(int data){
    struct node *newnode,*last;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

  last = head->prev;

    newnode->next = head;
    newnode->prev =last;

    last->next = newnode;
    head->prev = newnode;

    head = newnode;
}

void display(){
```

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

main.c

Share Run Output

```c
void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertStart(1);

    printf("After inserting: ");
    display();

    return 0;
}
```

```
After inserting: 1 10 20 30

=== Code Execution Successful ===
```

Insertion at end

main.c

Share Run Output

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning()
{
    struct node *first, *second, *last;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    last->data = 30;

    head = first;
```

```
After inserting at end: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

Output

```
    head = first;

    first->next = second;
    second->prev = first;

    second->next = last;
    last->prev = second;

    last->next = head;
    head->prev = last;
}

void insertEnd(int data)
{
    struct node *newnode, *last;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    last = head->prev;

    newnode->next = head;
    newnode->prev = last;

    last->next = newnode;
    head->prev = newnode;
```

```
After inserting at end: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

Output

```
    head->prev = newnode;
}

void display()
{
    struct node *temp = head;

    if(head == NULL)
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    beginning();
```

```
After inserting at end: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

Output

```
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    beginning();

    insertEnd(40);

    printf("After inserting at end: ");
    display();

    return 0;
}
```

```
After inserting at end: 10 20 30 40

=== Code Execution Successful ===
```

Insertion at position

main.c

Share Run

Output

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning()
{
    struct node *first,*second,*last;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    last->data = 30;

    head = first;

```

```
After inserting at position: 10 15 20 30

=== Code Execution Successful ===
```

main.c

Share Run

Output

```
    head = first;

    first->next = second;
    second->prev = first;

    second->next = last;
    last->prev = second;

    last->next = head;
    head->prev = last;
}

void insertPosition(int data,int pos)
{
    struct node *newnode,*temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    temp = head;

    for(i=1;i<pos-1;i++)
    {
        temp = temp->next;
```

```
After inserting at position: 10 15 20 30

=== Code Execution Successful ===
```

main.c

Share Run

Output

```
        temp = temp->next;
    }

    newnode->next = temp->next;
    newnode->prev = temp;

    temp->next->prev = newnode;
    temp->next = newnode;
}

void display()
{
    struct node *temp = head;

    if(head == NULL)
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
```

```
After inserting at position: 10 15 20 30

=== Code Execution Successful ===
```

```
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    beginning();

    insertPosition(15,2);

    printf("After inserting at position: ");
    display();

    return 0;
}
```

```
After inserting at position: 10 15 20 30

=== Code Execution Successful ===
```

Deletion at beginning

main.c — Share — Run — Output

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning()
{
    struct node *first,*second,*last;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    last->data = 30;

    head = first;

```

```
After deleting at beginning: 20 30

=== Code Execution Successful ===
```

main.c

Share Run

```
    head = first;

    first->next = second;
    second->prev = first;

    second->next = last;
    last->prev = second;

    last->next = head;
    head->prev = last;
}

void deleteBeginning()
{
    struct node *temp,*last;

    if(head == NULL)
    {
        printf("List is empty");
        return;
    }

    temp = head;
    last = head->prev;

    head = head->next;
```

Output

```
After deleting at beginning: 20 30

=== Code Execution Successful ===
```

main.c

Share Run

```
    temp = head;
    last = head->prev;

    head = head->next;
    head->prev = last;
    last->next = head;

    free(temp);
}

void display()
{
    struct node *temp = head;

    if(head == NULL)
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
```

Output

After deleting at beginning: 20 30

=== Code Execution Successful ===

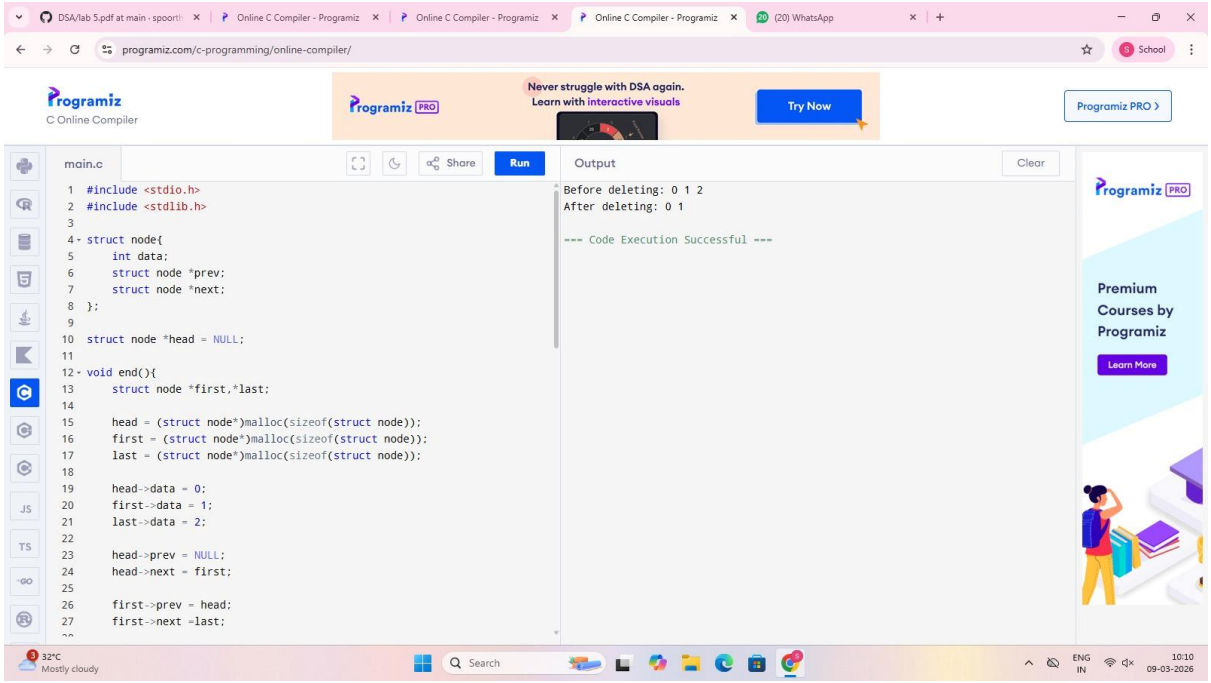
main.c

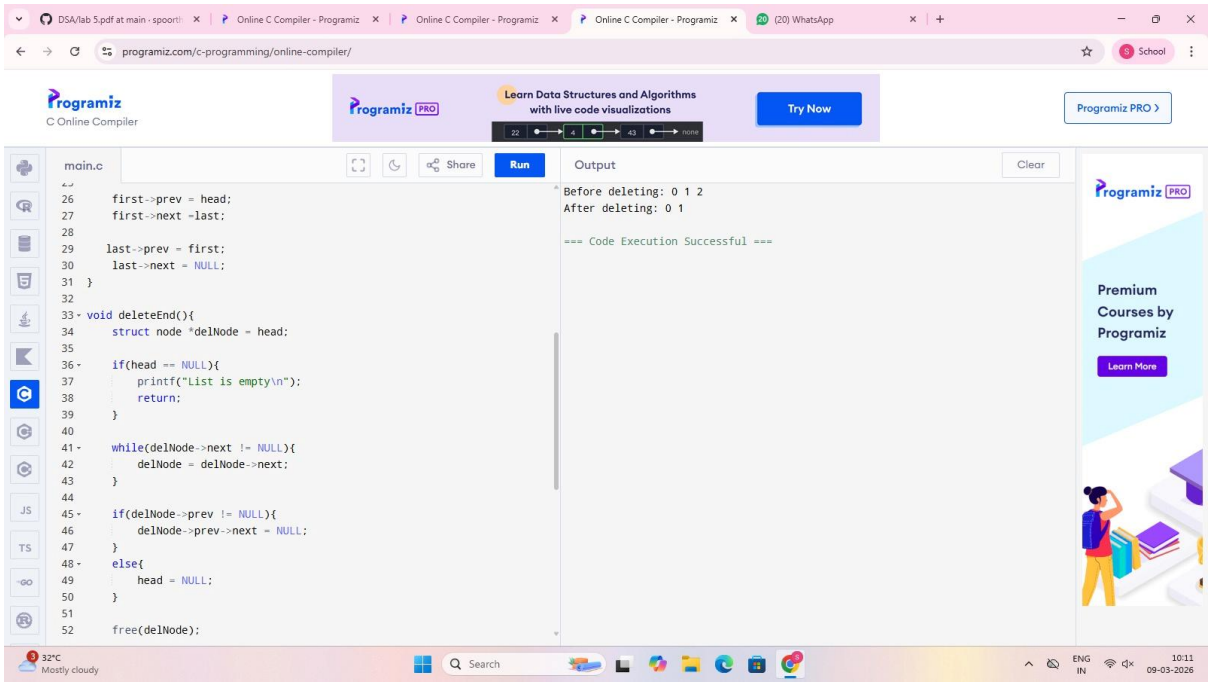
Share | Run | Output

```
    {
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main()
{
    beginning();

    deleteBeginning();

    printf("After deleting at beginning: ");
    display();

    return 0;
}
```

```
After deleting at beginning: 20 30

=== Code Execution Successful ===
```

Deletion at End

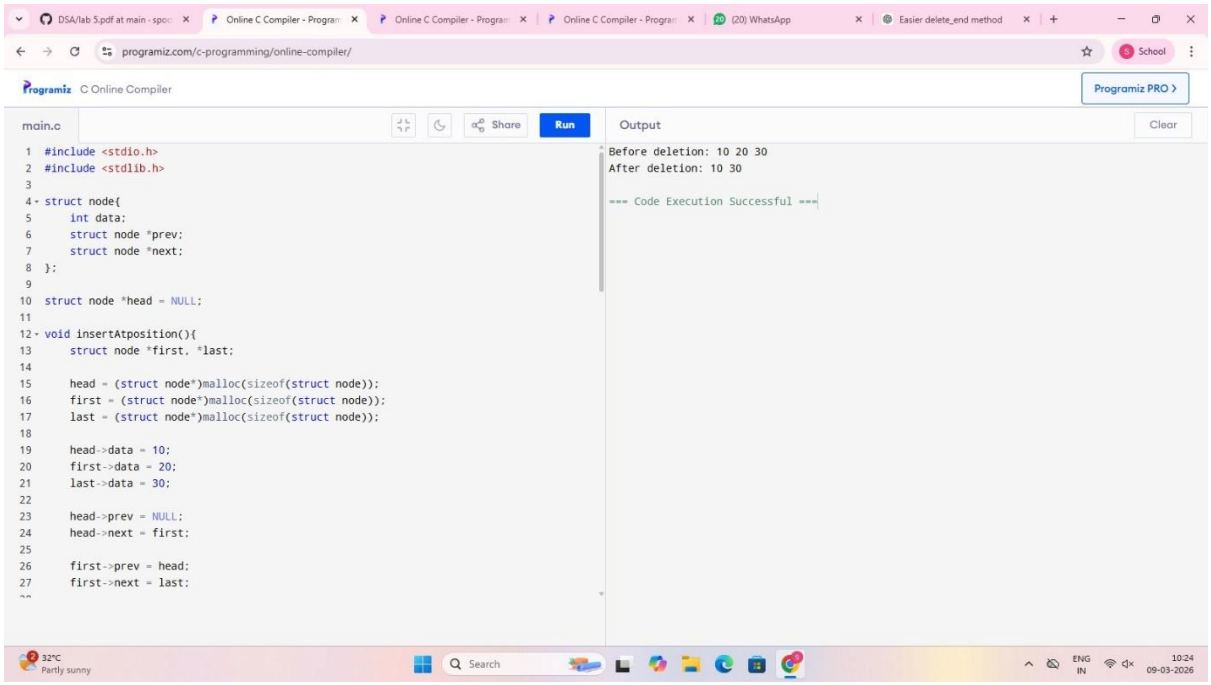





```c
    first->prev = head;
    first->next =last;

   last->prev = first;
    last->next = NULL;
}

void deleteEnd(){
    struct node *delNode = head;

    if(head == NULL){
        printf("List is empty\n");
        return;
    }

    while(delNode->next != NULL){
        delNode = delNode->next;
    }

    if(delNode->prev != NULL){
        delNode->prev->next = NULL;
    }
    else{
        head = NULL;
    }

    free(delNode);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    end();

    printf("Before deleting: ");
    display();

    deleteEnd();

    printf("\nAfter deleting: ");
    display();

    return 0;
}
```

```
Before deleting: 0 1 2
After deleting: 0 1

=== Code Execution Successful ===
```

Deletion at position

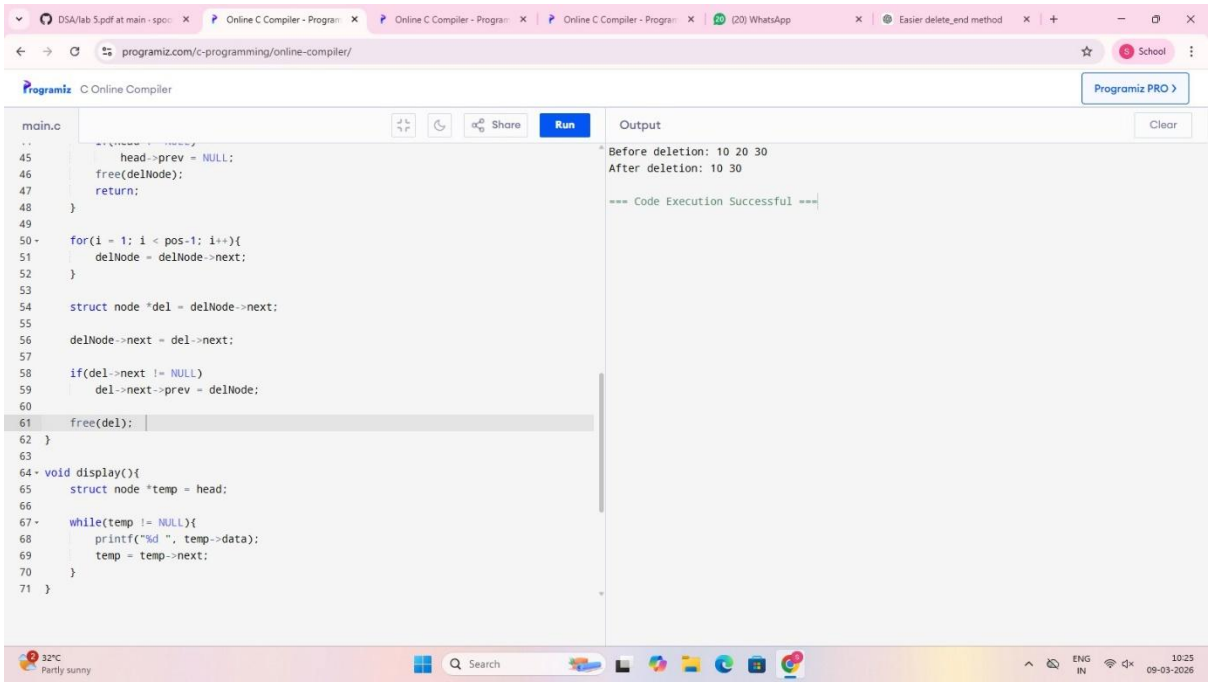


```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void insertAtposition(){
    struct node *first, *last;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));

    head->data = 10;
    first->data = 20;
    last->data = 30;

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = last;
```

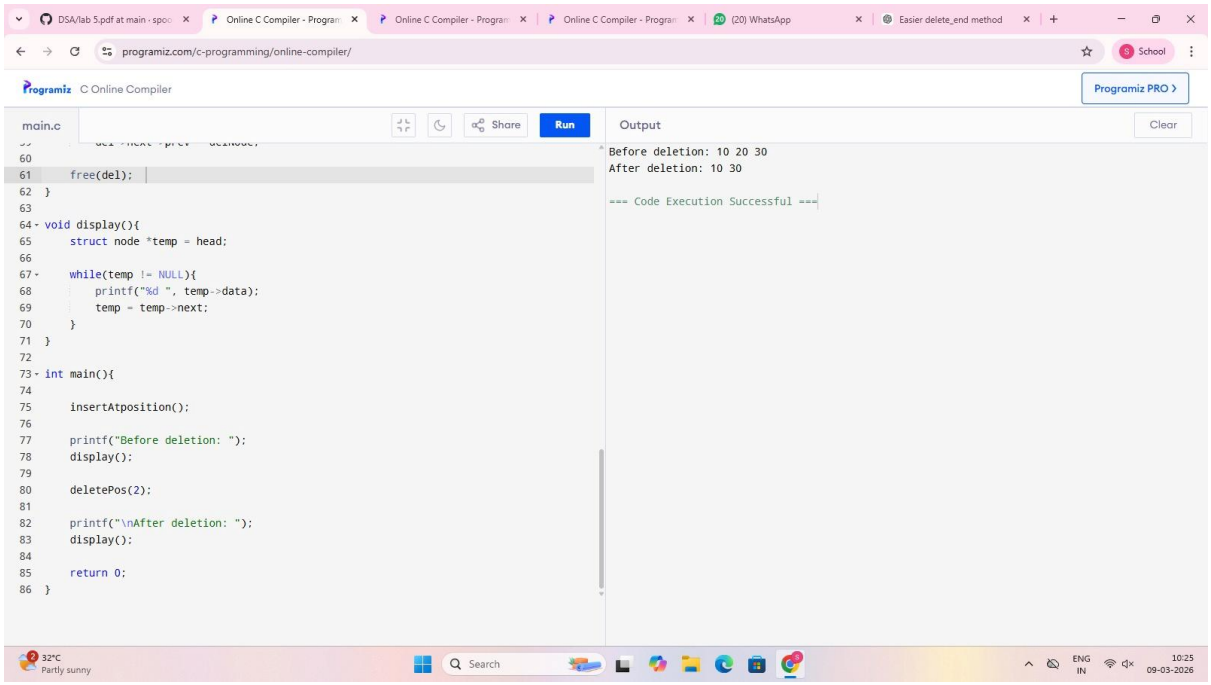
```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

```c

    last->prev = first;
    last->next = NULL;
}

void deletePos(int pos){
    struct node *delNode = head;
    int i;

    if(head == NULL){
        printf("List is empty\n");
        return;
    }

    if(pos == 1){
        head = head->next;
        if(head != NULL)
            head->prev = NULL;
        free(delNode);
        return;
    }

    for(i = 1; i < pos-1; i++){
        delNode = delNode->next;
    }

    struct node *del = delNode->next;

```

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```




```
45          head->prev = NULL;
46      free(delNode);
47      return;
48  }
49
50  for(i = 1; i < pos-1; i++){
51      delNode = delNode->next;
52  }
53
54  struct node *del = delNode->next;
55
56  delNode->next = del->next;
57
58  if(del->next != NULL)
59      del->next->prev = delNode;
60
61  free(del);
62 }
63
64 void display(){
65      struct node *temp = head;
66
67      while(temp != NULL){
68          printf("%d ", temp->data);
69          temp = temp->next;
70      }
71 }
```




```
60
61      free(del);
62  }
63
64  void display(){
65      struct node *temp = head;
66
67      while(temp != NULL){
68          printf("%d ", temp->data);
69          temp = temp->next;
70      }
71  }
72
73  int main(){
74
75      insertAtposition();
76
77      printf("Before deletion: ");
78      display();
79
80      deletePos(2);
81
82      printf("\nAfter deletion: ");
83      display();
84
85      return 0;
86  }
```